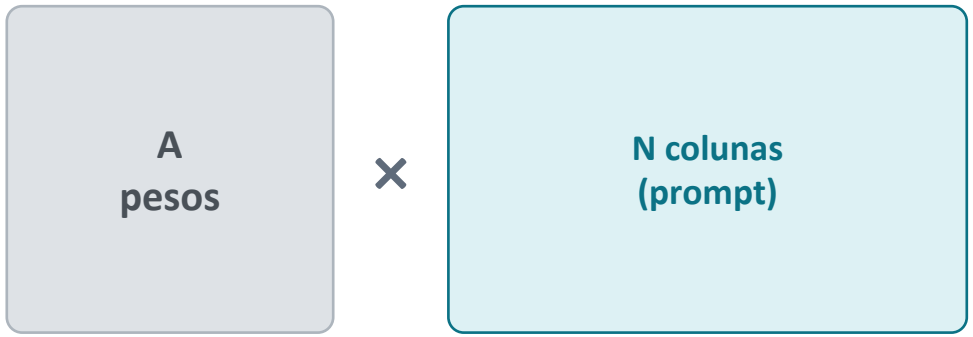
# O que muda para uma GPU, uma CPU e uma NPU quando o gargalo é ler memória em vez de fazer conta?

**FAZER CONTA · FLOPs** vs **LER MEMÓRIA · bytes**

Spoiler: o que manda é a banda e quantos bytes cada token precisa. O que muda em cada chip é como ele lida com isso.

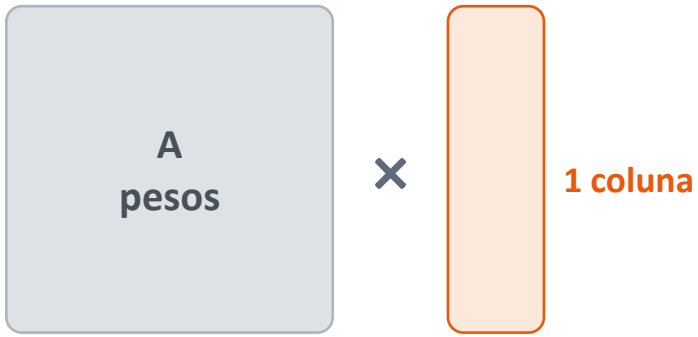
# Prefill é GEMM, decode é GEMV: o mesmo peso, dois usos

**PREFILL · GEMM**




Cada peso lido serve N tokens: muita conta por byte.

**DECODE · GEMV**




Cada peso lido serve 1 token: pouca conta por byte.

## ≈ 1 FLOP/byte

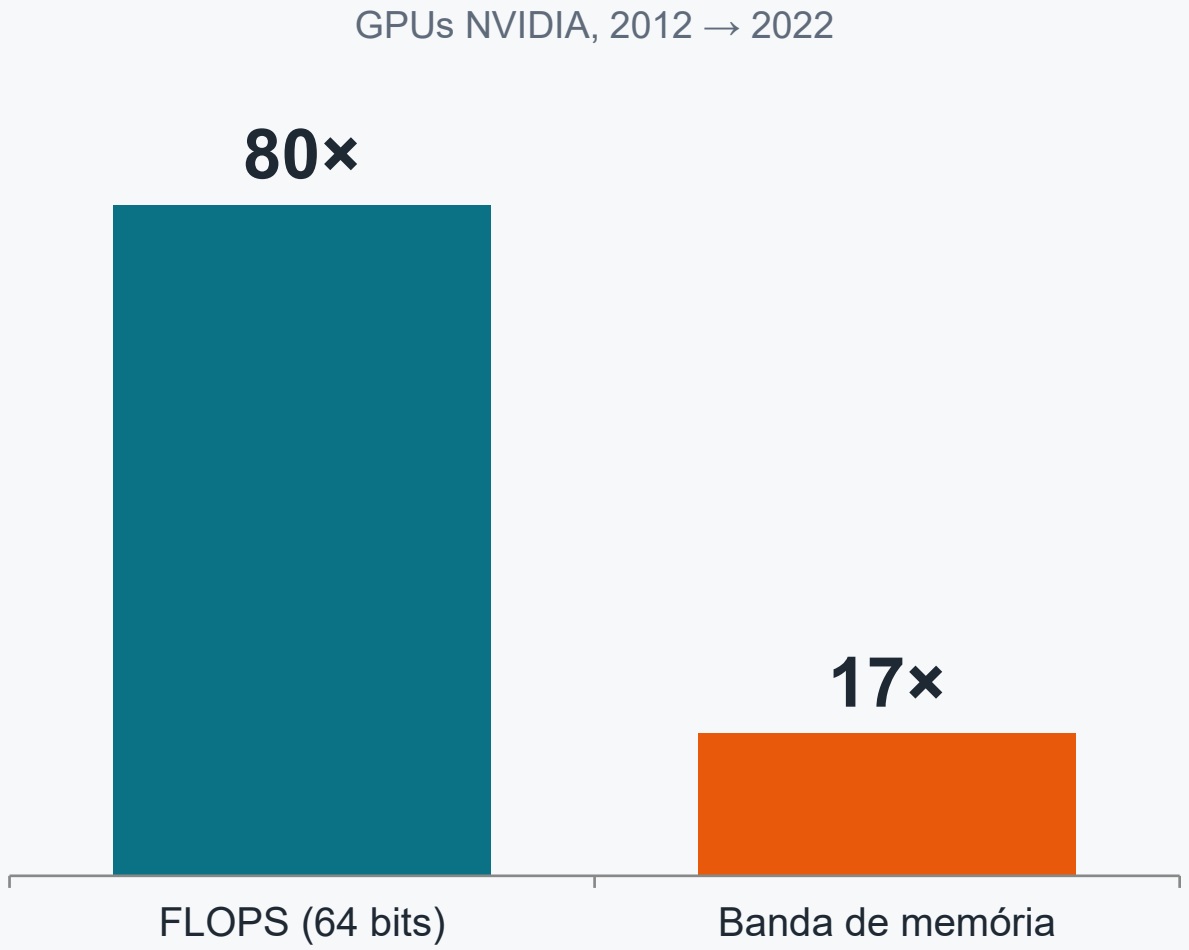
decode com batch 1 e pesos em fp16

## ≈ 4 FLOP/byte

a mesma conta com pesos em int4: menos bytes, mais conta por byte

Conta: 2 FLOPs por peso ÷ bytes por peso (ignora ativações e KV cache). NVIDIA, Matrix Multiplication Background: GEMV é sempre limitado por memória.

# O memory wall: a conta cresceu 80×, a banda só 17×




**×1,35**

custo do HBM por GB e por GB/s, 2023 → 2025

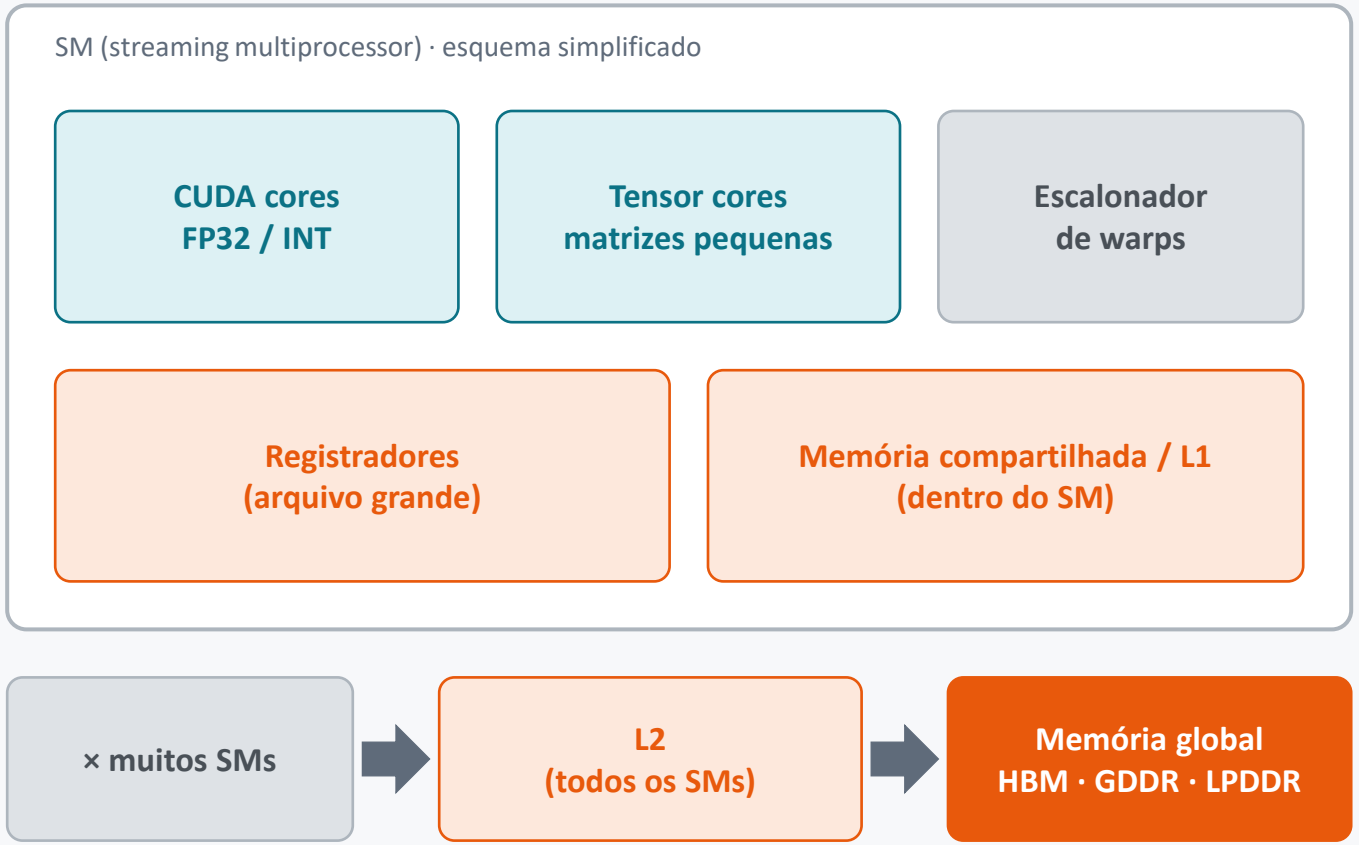
**×0,54 · ×0,45**

custo por GB e por GB/s do DDR4, 2022 → 2025

Mantenha o qualificador: são FLOPS de 64 bits, não a precisão usada em inferência. Para os autores, o hardware atual (chip enorme, muito FLOPS, muito HBM) não casa com o decode.

Fonte: Ma & Patterson, Challenges and Research Directions for LLM Inference Hardware, arXiv 2601.05047v3 (jan/2026; aceito no IEEE Computer).

# GPU I: muitos SMs; a latência se esconde trocando de warp




**Warp = 32 threads executando a mesma instrução (SIMT).**

**Enquanto um warp espera a memória, o SM executa outro: a latência é escondida, não eliminada.**

**Exemplo: Jetson Orin Nano Super**

1024 CUDA cores · 32 tensor cores · 8 GB LPDDR5 · 102 GB/s (modo MAXN_SUPER)

Esquema simplificado; as quantidades por SM variam por arquitetura. Fontes: NVIDIA CUDA Programming Guide (warp); ficha técnica do Jetson Orin Nano Super.

# GPU II: a pirâmide de memória, e o decode vive no último degrau




topo: menor, mais rápido, mais caro por byte · base: maior, mais lento, mais barato

**PREFILL · reuso**

Traz um bloco de pesos para o topo da pirâmide e o reaproveita em N tokens.

**DECODE · sem reuso**

Todo peso sai da base a cada token: a banda do último degrau é o teto.

**Se o gargalo é mover bytes, dá para acelerar movendo menos → kernels e fusão.**

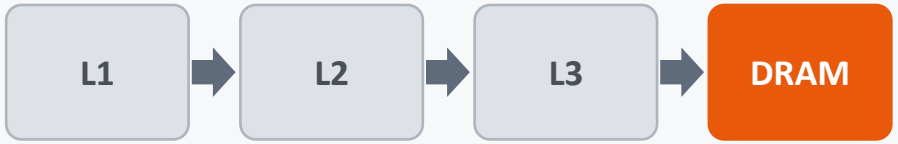
Esquema conceitual; capacidades e larguras de banda variam por GPU (consulte deviceQuery ou o whitepaper da sua GPU).

# CPU: o cache não ajuda quem lê cada peso uma vez

**5%**

da capacidade de cálculo medida é usada no GEMV (N=1) da minha CPU

11,7 de 232 GFLOP/s · 23,4 GB/s de banda

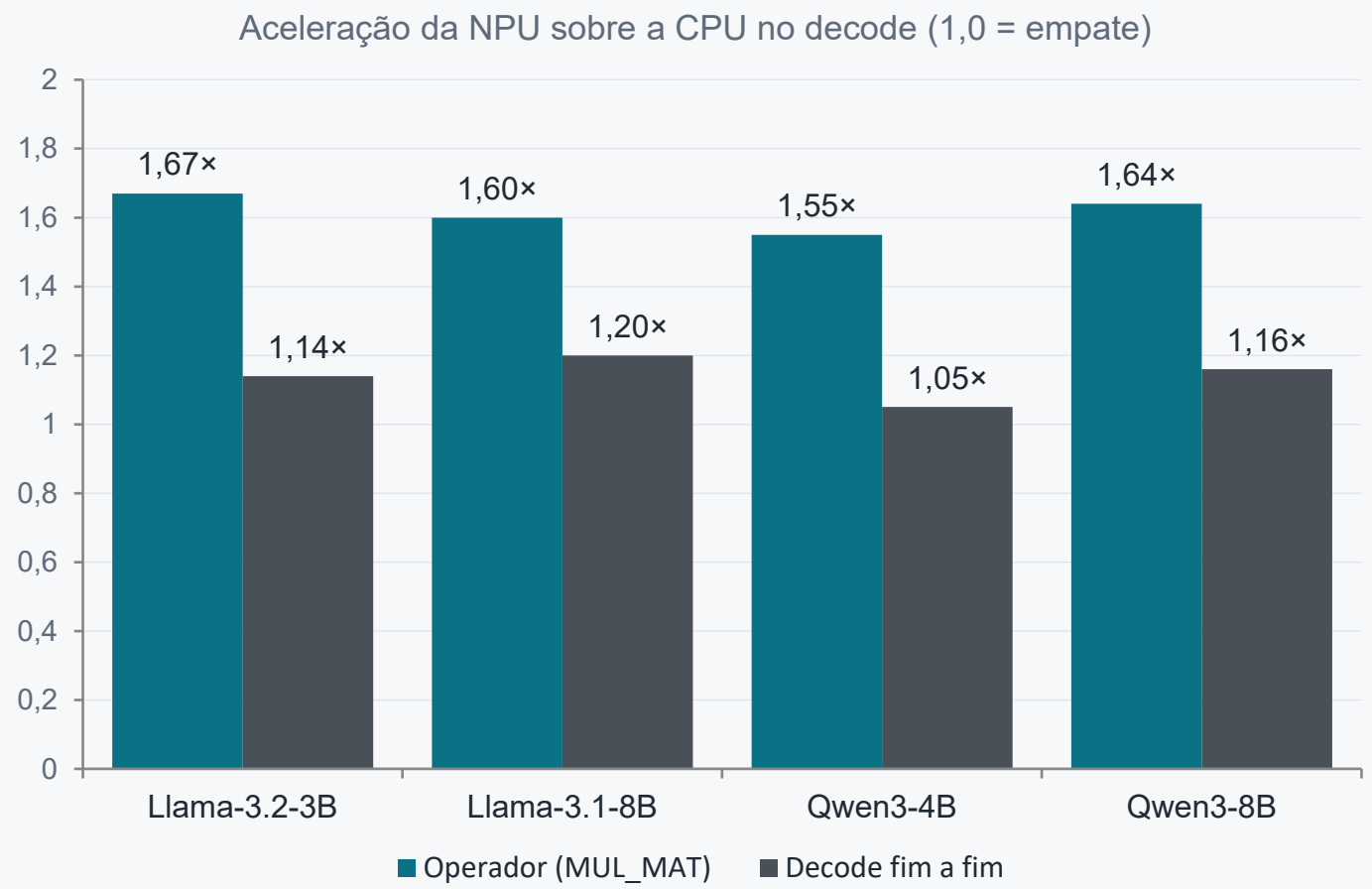



Caches gerenciados por hardware: ótimos com reuso, inúteis para dados lidos uma vez.

Fontes: arXiv 2605.27435 (mai/2026) e 2607.05475 (jul/2026); medição própria com gemm_vs_gemv.py (máquina pessoal, detalhes no slide 11).

# NPU I: ganha no operador, perde no sistema

Snapdragon 8 Gen 3 · llama.cpp (backend Hexagon) · Q4_0 · CPU de referência com 6 threads




**8–22×**

tempo de chamada vs execução em operadores leves (RMS_NORM, ADD)

**≈ 26–31%**

do tempo do caminho NPU foi fallback para a CPU (ex.: FLASH_ATTN_EXT)

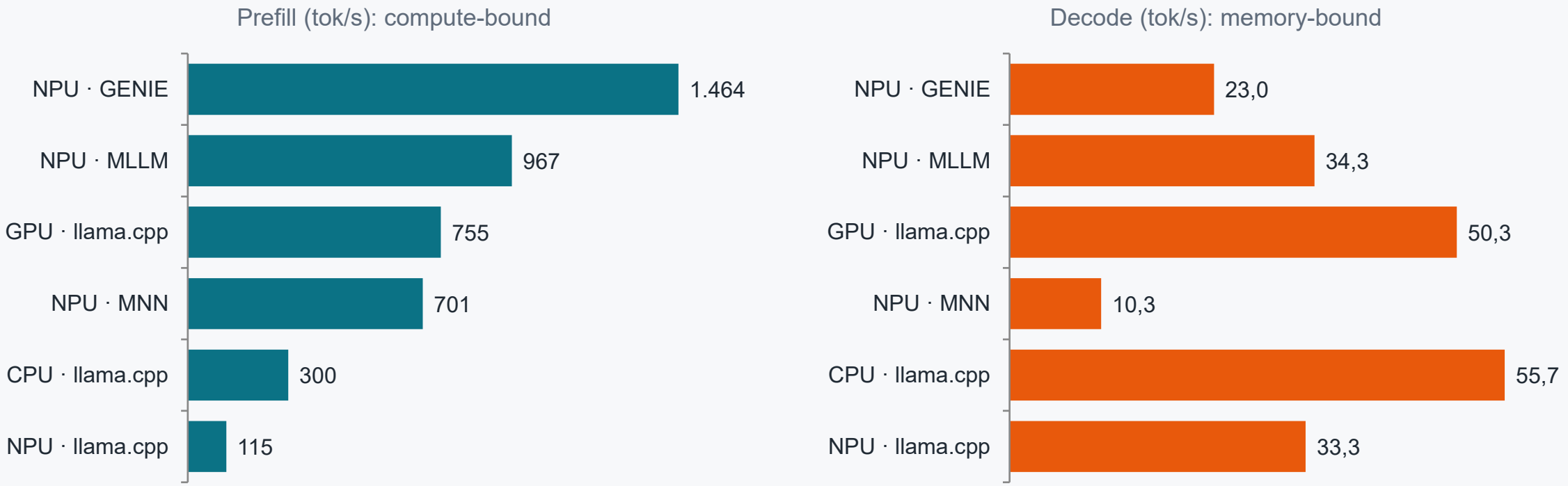
**CPU ganha o prefill**

1,27–1,62× mais rápida no operador

Fonte: When NPUs Are Not Always Faster, arXiv 2605.27435 (mai/2026). O 26–31% é conta minha a partir da Tabela V (fallback ÷ caminho NPU).

# NPU II: no mesmo chip, o framework muda o prefill em 12,7×

OnePlus 15 · Qwen2.5-1.5B · pesos em 4 bits · 256 tokens




**Prefill: GENIE 1463,7 vs llama.cpp 115,1 tok/s na mesma NPU (12,7×). Decode: a CPU lidera e as NPUs ficam atrás.**

Fonte: Is Your NPU Ready for LLMs?, arXiv 2607.05475 (jul/2026), Tabela 5. MLLM usa pesos 4 bits e ativações 16 bits (w4a16). Preprint.

# TPU, LPU e LPX: a mesma resposta (mais SRAM) para o mesmo gargalo

**TPU 8i · Google, abr/2026**

## 384 MB

**de SRAM por chip, 3× o Ironwood**

A geração 8 separa chip de treino (8t) e de inferência (8i).

**Groq 3 LPU · Nvidia, mar/2026**

## 500 MB

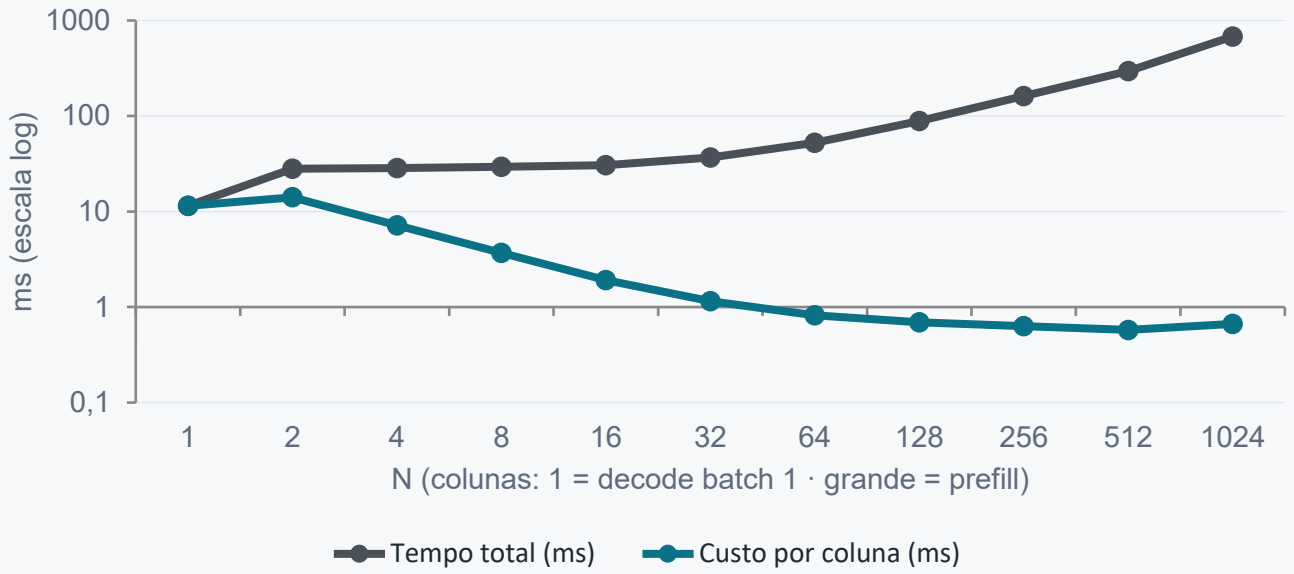
**de SRAM e 150 TB/s por chip**

256 LPUs por rack LPX.

O limite é a capacidade: os pesos ocupam a SRAM e sobra pouco para o KV cache quando o batch cresce.

Fontes: CNBC (22/04/2026) e blog do Google; NVIDIA Developer Blog (Groq 3 LPX); SemiAnalysis; The Register; Ma & Patterson (arXiv 2601.05047).

# O que eu medi: 1 coluna custa 11,5 ms; com 512, cada coluna custa ~20× menos




| N | ms total | ms/coluna | GFLOP/s | % do pico |
|---|---|---|---|---|
| 1 | 11,49 | 11,49 | 11,7 | 5% |
| 16 | 30,58 | 1,91 | 70,2 | 30% |
| 64 | 52,54 | 0,82 | 163,5 | 70% |
| 512 | 295,63 | 0,58 | 232,5 | 100% |

**5%** do pico de cálculo usado no GEMV (N=1)

**≈ 20×** menor custo por coluna com N=512 que com N=1

**20–30** N de virada de memória para conta (estimado: 20; o tempo sobe a partir de N=32)

Conta: modelo de 0,8 GB ÷ 23,4 GB/s ≈ 29 tok/s (teto; a medir com llama-bench).

Máquina pessoal (não é a do time) · Intel família 6 modelo 154 · 12 threads lógicas · Windows 11 · Python 3.14.8 · NumPy 2.5.3 + OpenBLAS 0.3.34 · fp32 · A = 8192×8192 (256 MiB) · 15 execuções por ponto (mediana; IQR 2,5–16,6%) · RAM e plano de energia não registrados · script: gemm_vs_gemv.py

# A resposta

Hipótese até a medição na máquina do time

**CPU**

O QUE MUDA

**O cache deixa de ajudar e a conta fica ociosa.**

EVIDÊNCIA

Minha CPU: o GEMV usa 5% do pico de cálculo. Uma coluna custa 11,5 ms; com 512, cada uma custa 0,58 ms.

**GPU**

O QUE MUDA

**Sobra conta: só um batch maior usa o chip.**

EVIDÊNCIA

NVIDIA: GEMV é sempre limitado por memória. FLOPS ×80 contra banda ×17 (2012→2022). Ainda sem medição própria.

**NPU**

O QUE MUDA

**O ganho no operador não vira ganho no sistema; o framework decide.**

EVIDÊNCIA

Maio: 1,6× no operador, 1,1× fim a fim. Julho: o prefill varia 12,7× entre frameworks no mesmo chip.

Medido só na CPU (máquina pessoal). GPU e NPU: evidência da literatura.  Perguntas?